
Topic 4 – Async Research Assistant

zerobias

Ibrahim Mammadov • Fidan Allahverdiyeva • Fatma Mammadova

Final Project Defense – May 23, 2026

https://github.com/IBRAHIM102005/AIENG_FinalProject_Topic4

The problem in one sentence

What the system does

A user types a research question; the system queries Wikipedia, arXiv, and web search *in parallel* and returns a concise, cited answer in seconds.

Inputs

- ▶ Question string (up to 1000 chars)
- ▶ Optional: -sources, -limit, -timeout, -concurrency, -no-cache, -offline

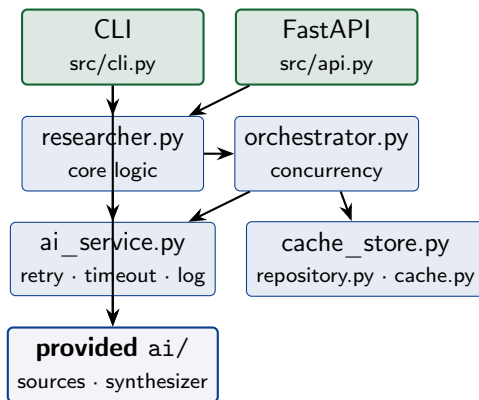
Outputs

- ▶ Synthesised answer with [N] inline citations
- ▶ Source list: title + URL per reference
- ▶ Session record persisted in SQLite

Entry points: `python -m researcher ask "..."` | `POST /api/research` |
Browser UI at `localhost:8000`

Architecture

Module map. The boundary between the *provided* ai/ package and *our* SE layer is shown explicitly.



Swapping Anthropic → OpenAI: change 2 env vars, zero source files.

Three design decisions we can defend

1. **asyncio over threads**

Source fetches are I/O-bound; `asyncio.gather` overlaps all three waits on one OS thread with no GIL contention. Threads were rejected because they add locking complexity for zero gain on pure I/O.

2. **Pydantic Settings over manual `os.getenv` parsing**

Type-safe, validated at startup, and bridges values into `os.environ` for the provided `ai/` package automatically. Trade-off: one extra dependency (`pydantic-settings`).

3. **SQLite over PostgreSQL**

Zero-ops, file-based, sufficient for a single-process research tool. We would switch to `asyncpg` + PostgreSQL the moment we need multi-worker deployment or horizontal scaling.

Expect to be asked

“Why not threads / bare `dotenv` / Postgres from the start?”

Concurrency: the benchmark

Workload: 3 sources, 1 question, offline fake fetchers (simulated latencies: Wikipedia 0.45 s, arXiv 0.65 s, web 0.55 s), caches cleared, Python 3.12.9 on AMD Ryzen 7 Windows 11.

	Sequential	Concurrent (sem=5)
Wall-clock (s)	1.680	0.653
Speedup	1.0×	2.57×
Provider 429s	0	0

Bottleneck named: Event-loop scheduling overhead + imperfect latency alignment; theoretical max is 3×

What didn't reach $N\times$: The blocking `synthesize` call (run in thread-pool executor) adds 1–4 s LLM latency that cannot be parallelised.

Reproduce: `python scripts/bench.py` (offline, no API keys needed).

Robustness: one concrete failure mode

Scenario: Wikipedia returned HTTP 403 in live integration because no User-Agent header was set—silent in offline tests, breaking every live run.

How we surfaced it: Observed naturally on first live CLI run; confirmed by passing an empty-headers `httpx.AsyncClient` in a test.

What the system does:

- ▶ **Retry:** tenacity exponential backoff (3 attempts, 1–30 s) on `TimeoutError`, `ConnectionError`, `OSError`.
- ▶ **Degrade:** after 3 failed attempts, `SourceResult.degraded()` is returned—pipeline continues with arXiv + web.
- ▶ **Log:** `WARNING source_fetch_failed source=wikipedia elapseded=... error=403 Forbidden`
- ▶ **User sees:** normal cited answer + note “wikipedia failed: Client error '403 Forbidden'”.

Fix: injected a single shared `httpx.AsyncClient` (with User-Agent) from CLI/API through the full call stack.

Live demo (or scripted demo)

What we will run

Offline demo answering all 5 example questions from `data/research_questions.json`, showing parallel fetch timings and inline citations per answer.

Demo command (works inside Docker):

```
$ docker run --env-file .env finalproj \  
  python scripts/demo.py --limit 5
```

- ▶ Each answer shows [1], [2], [3] inline markers with source title + URL beneath.
- ▶ Parallel wall-clock is printed per question; sequential equivalent would be $\approx 2.6\times$ slower.
- ▶ Pass `-live` to call real providers (API keys required).

Pre-recorded screencast available as backup. See `artefacts/demo_offline.txt` for sample output.

Demo Screenshot

TOPIC 4 DEMO

5.26s

Async Research Assistant

Research question

Ask

Sources

☒ Wikipedia ☒ arXiv ☒ Web

Limit

☐ Offline demo ☐ No cache

Answer

Quantum computing is a multidisciplinary field combining aspects of computer science, physics, and mathematics [4]. It represents a new approach to calculation that utilizes principles of fundamental physics [5] and quantum mechanics [4]. This technology is based on concepts such as the superposition of matter and quantum entanglement [6]. Quantum computers are designed to solve extremely complex problems very quickly [5], enabling them to complete tasks that would be nearly impossible for ordinary computers [6].

Timings

- wikipedia: 0.039s
- arxiv: 0.516s
- web: 1.585s

Source Notes

- wikipedia: Wikipedia search failed: Client error '403 Forbidden' for url 'https://en.wikipedia.org/w/api.php?action=opensearch&search=What+is+quantum+computing%3F&limit=3&namespace=0&format=json' For more information check: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/403>

References

- [\[4\] What is Quantum Computing? - AWS](#)
web - Quantum computing is a multidisciplinary field comprising aspects of computer science, physics, and mathematics that utilizes quantum mechanics to solve
- [\[5\] What is quantum computing? - McKinsey](#)
web - Quantum computing is a new approach to calculation that uses principles of fundamental physics to solve extremely complex problems very

Testing I

Coverage & shape

- ▶ Total coverage: **78 %** (`pytest -cov=src`)
- ▶ Provided ai/ smoke tests: **passing**
- ▶ 184 tests total; all run offline
- ▶ AI module fully mocked via FakeLLM + AsyncMock

Three tests worth calling out

- ▶ `test_research_question_returns_`
(`test_end_to_end.py`) — full pipeline, partial failure
- ▶ `test_fetch_selected_sources_degr`
(`test_concurrency.py`) — gather degrades cleanly
- ▶ `test_cache_entry_expired_return`
(`test_cache_entry.py`) — caught a naive-datetime timezone bug

Testing II

Tooling

`pytest + pytest-asyncio + pytest-cov + respx / unittest.mock.`
mypy run once; results reported in the written report.

Limitations & what we'd do next

Where the system is brittle today

- ▶ SQLite is single-writer; a multi-worker deployment needs PostgreSQL + asyncpg.
- ▶ No token-per-minute rate limiting—heavy prompts can hit the LLM TPM ceiling even at low concurrency.
- ▶ Cache staleness undetected: arXiv abstracts served stale until TTL expires, no ETag check.
- ▶ synthesize blocks a thread-pool slot for 1–4 s; streaming LLM calls would fix this.

Given another week we would

1. Add OpenTelemetry spans around every external call for production observability.
2. Load-test with k6 to find the real concurrency ceiling under simultaneous `/api/research` requests.

Team contributions

Member	Primary ownership	Commits
Ibrahim Mammadov	src/config.py, src/services/ai_service.py (retry, failover), src/services/cache.py, CI workflows	~34%
Fidan Allahverdiyeva	src/concurrency/orchestrator.py, src/core/researcher.py, src/cli.py, src/api.py, src/web/, scripts/, tests/test_concurrency.py, test_end_to_end.py, test_api.py	~33%
Fatma Mammadova	src/storage/ (cache_store + repository), Dockerfile, README.md, test suite	~33%

We can all answer:

Any of us can walk through any module on demand. Pick a file — we'll defend it.

AI tool use: Claude scaffolded initial retry logic and storage DDL; all backoff parameters, SQL, and business logic were reviewed and adapted by the team.

Questions?

zerobias

https://github.com/IBRAHIM102005/AIENG_FinalProject_Topic4

"We can walk through any file in the repo."